

Web Systems Development

Week 4 · CSS: the cascade, the tools, the frameworks



September 15, 2026

AGENDA

HM: Correction and grading issues

AGENDA

HW: Grading and corrections

- Your grade is posted as an **issue on your repo**
- There will be grading issues; we handle them **like developers**
- Concerned about a grade? **Investigate on your own first** — check whether what you missed is clear
- If you still find something, **describe the issue as clearly as you can** and tag @jkuruzovich and @JinzhaoKang
- **Goal:** grading done in one week; issues addressed within one week

Readings



Keith Grant, *CSS in Depth*, 2nd ed., ch. 9

Modular CSS and scope. Why the cascade makes every rule global, then the lineage of answers: modules, BEM, utilities, @scope. Ch. 1 if you cannot say why .nav a beats a.link. [RPI library](#), [O'Reilly](#).

Adam Wathan, CSS Utility Classes and “Separation of Concerns” (2017)

[The essay](#) that became Tailwind. He starts as a believer in semantic names and refactors his way off it. Read it to argue with it.

[Optional: Gallagher, About HTML Semantics and Front-End Architecture \(2012\)](#)

The position Wathan is arguing against, from the person who made it best.

The question the readings disagree on: should a class name say what a thing is, or what it looks like?

Tailwind for CSS: Two VS Code Packages

Extension: Tailwind CSS IntelliSense ✕



Tailwind CSS IntelliSense

Tailwind Labs  tailwindcss.com |  14,600,920 |  (118)

Intelligent Tailwind CSS tooling for VS Code

Disable ▾ Uninstall ▾ ☒ Auto Update 

DETAILS FEATURES CHANGELOG



Live Preview Preview

Microsoft  microsoft.com |  13,622,761 |  (79)

Hosts a local server in your workspace for you to preview your webpages on.

Switch to Pre-Release Version Disable ▾ Uninstall ▾

☒ Auto Update 

Use Tailwind and shadcn/ui together

Tailwind is the styling system; shadcn/ui adds reusable component code you own and customize.



Tailwind CSS

UTILITY-FIRST CSS FRAMEWORK

- Compose designs with utility classes directly in your markup.
- Use responsive and state variants such as md:, hover:, and focus:.
- Generate static CSS from the classes detected in your project files.
- Best for layout, spacing, color, typography, and custom UI.

```
flex items-center gap-4 p-6
```



shadcn/ui

UI.SHADCN.COM

- Add customizable components to your project as source code.
- Own and edit the component code instead of using a black-box package.
- Style components with Tailwind and theme them with CSS variables.
- Best for buttons, dialogs, forms, tables, and navigation.

```
pnpm dlx shadcn@latest add button
```

RECOMMENDED WORKFLOW

Tailwind for design primitives → shadcn/ui for polished components → customize both in your codebase.

ITWS 2110 · WEEK 4 · FRIDAY

Tailwind's Flexbox, Grid, and Breakpoints

Three tools do almost all of the layout on the modern web. One puts things in a row, one puts things in a grid, and one changes its mind when the screen gets wider.

Three questions, three answers

Is it a row?

One direction. A header with a title at one end and links at the other. Items size themselves to their content.

`flex`

Is it a grid?

Two directions at once. Six cards in even columns and rows, every column the same width whatever is inside it.

`grid`

Does it change?

The same page at 375 px and at 1280 px are different layouts. A prefix says from which width the change applies.

`md:`

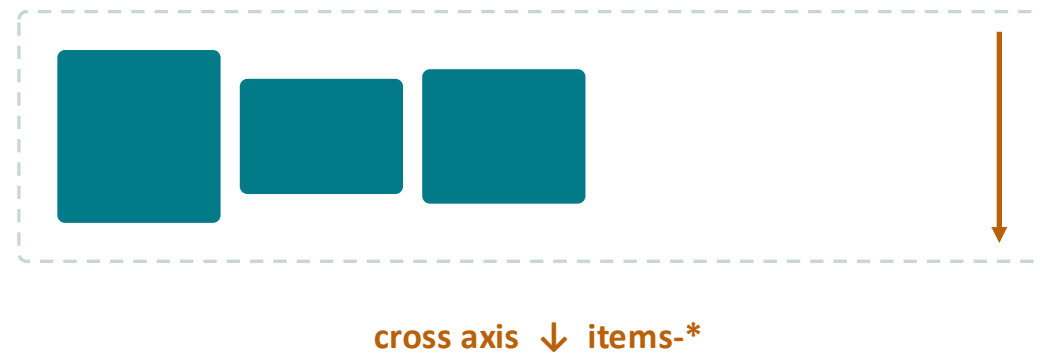
Tailwind's class names are the CSS property names. Learning the classes and learning the CSS underneath is one activity, not two.

The parent decides; the children go along with it

```
<header class="flex">  ← the parent becomes a flex container  
  <h1>Title</h1>      ← this is now a flex item  
  <nav>Links</nav>    ← so is this  
</header>
```

You never put a class on the thing you want to move. You put it on its parent, and the parent arranges everything inside it.

Two axes, two properties



This is the single most common mistake: styling the child when the answer belongs on the parent.

justify-* spreads items on the main axis



justify-start
the default



justify-center
grouped, centred



justify-between
ends pushed apart

Tailwind	The CSS it compiles to	Where you want it
justify-between	justify-content: space-between	title left, links right — HW3 Task 1
justify-center	justify-content: center	a centred group of buttons
items-center	align-items: center	everything on one middle line
gap-4	gap: 1rem	space between items, no margins needed

gap replaced the margin tricks people used for twenty years. Use it.

items-* handles the other direction



items-start

tops line up



items-center

middles line up



items-stretch

the default: equal height

Why items-center is the one you reach for

- A header holds a big bold title and three small links. Left alone they sit on their own baselines and the row looks accidental.
- items-center puts every item on one invisible middle line, whatever its height. It is what makes a nav bar look deliberate.
- flex-col turns the row into a column — and then the two axes swap: justify-* now runs vertically and items-* runs horizontally.

Both properties live on the container. Neither one is ever a class on the thing you are trying to move.

The header, and the flex container inside it

```
<header class="flex justify-between items-center
            px-6 py-4 bg-white shadow">

  <h1 class="text-xl font-bold">My Course Page</h1>

  <nav class="flex gap-6">                ← flex again
    <a href="#">Home</a>
    <a href="#">Projects</a>
    <a href="#">About</a>
  </nav>

</header>
```

What it produces

My Course Page

Home Projects About

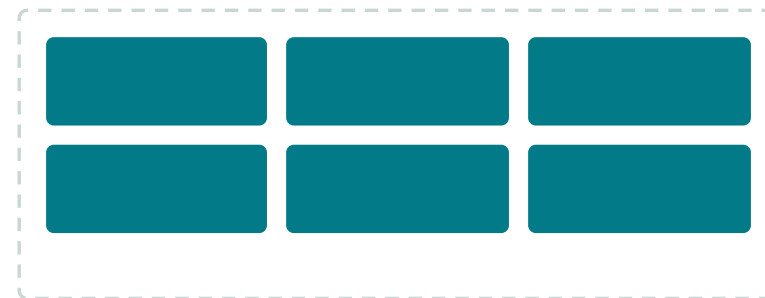
- The outer flex puts the title and the nav at opposite ends.
- The nav is a flex container too — that is what spaces the three links.
- Nesting flex inside flex is normal. You will do it constantly.

No widths, no floats, no positioning. Two flex containers and a gap.

You declare the columns; items fall into them

Flexbox asks the items how big they want to be. Grid tells them. That is the whole difference, and it is why an even card layout is a grid and a nav bar is not.

```
<section class="grid grid-cols-3 gap-4">
  <div>Card One</div>      ← row 1, column 1
  <div>Card Two</div>      ← row 1, column 2
  <div>Card Three</div>    ← row 1, column 3
  <div>Card Four</div>     ← row 2, column 1 (wraps by itself)
</section>
```

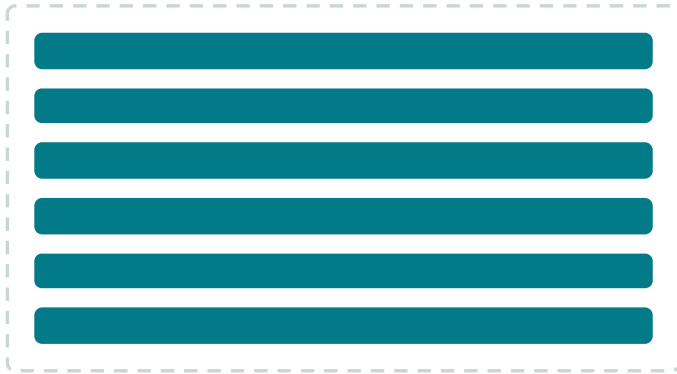


grid-cols-3

six items, three tracks

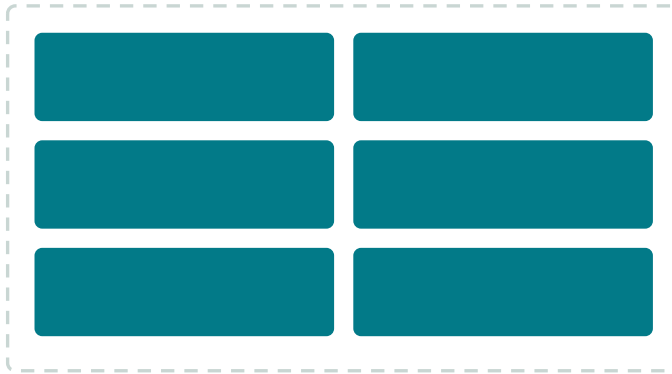
- You never say which cell an item goes in. You declare how many columns exist and the items flow into them in order.
- Every column is the same width regardless of its contents — which is exactly what a row of cards should do, and what flexbox will not do for free.
- gap-4 spaces both directions at once: between columns and between rows.

One property, three screen sizes



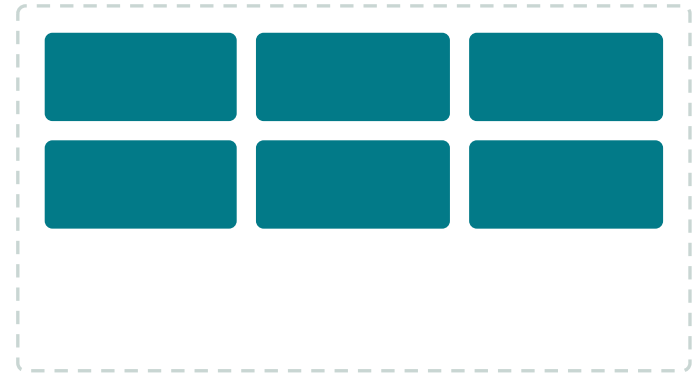
`grid-cols-1`

phone · no prefix



`md:grid-cols-2`

tablet · from 768



`lg:grid-cols-3`

laptop · from 1024

```
<section class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
```

Read it left to right as a sentence: one column, then two from md, then three from lg.

Same six divs, same HTML, three layouts. Nothing moved in the markup — only the column count changed. NOTE THIS IS A MOBILE FIRST DESIGN

SO WHICH ONE?

One dimension or two

Flexbox

Grid

Thinks in

one direction: a row or a column

two at once: rows and columns

Sizes items

from their content outwards

from the tracks you declared inwards

Wrapping

items keep their own widths

every column matches the others

Reach for it when

a bar, a toolbar, a row of buttons, anything with two ends

cards, galleries, dashboards, a page skeleton

In HW3

Task 1, the header

Task 2, the cards

They are not rivals and you do not have to choose globally. A grid of cards, where each card is a little flex column, is an ordinary page.

If you find yourself fighting one of them, you have probably picked the wrong one for that particular box.

A media query applies CSS when a condition matches

The browser checks a feature—usually viewport width—and turns a block of styles on or off.

THE PATTERN

```
@media (min-width: 768px) {  
  .cards {  
    grid-template-columns: 1fr 1fr;  
  }  
}
```

HOW TO READ IT

1

Check the condition

Is the viewport at least 768px wide?

2

Apply the rules if true

The cards switch from one column to two.

THE RESULT

Below 768px

**One-column
layout**



768px and above

**Two-column
layout**



In short: a media query is an if statement for CSS.

md: is a media query with a shorter name

You write

```
<div class="grid-cols-1 md:grid-cols-2">
```



Tailwind generates

```
.grid-cols-1 {  
  grid-template-columns: repeat(1, 1fr);  
}  
  
@media (width >= 48rem) { /* = min-width: 768px */  
  .md\:grid-cols-2 {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```

- There is no Tailwind magic here. A prefix is a media query, written for you, with the breakpoint already filled in.
- The unprefixed class has no media query at all — which is why it applies at every width, including wide ones.
- min-width, not max-width. That one word is what makes the whole system mobile-first.

If you can read the CSS on the right, you can write a media query by hand in any project, with or without Tailwind. That is the transferable part.

Describe the phone, then add what width affords

Prefix	Applies from	Think
(none)	0 px — every width	the phone, and the baseline everything else builds on
sm:	640 px	large phone, landscape
md:	768 px	tablet — the breakpoint you will use most
lg:	1024 px	laptop
xl:	1280 px	desktop monitor

This is a design decision, not a habit

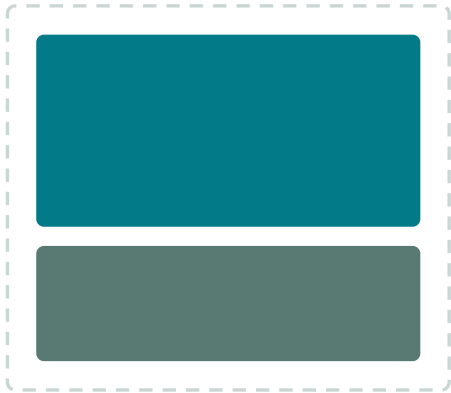
Tailwind's own documentation says it: "Tailwind uses a mobile-first breakpoint system." Every breakpoint is a min-width, on purpose.

So a phone layout — one column, things stacked — is mostly the browser's default, and the unprefixed version is nearly free. Each prefix then adds only what the extra width buys.

You never write "shrink this back down for phones." If you are, you started at the wrong end.

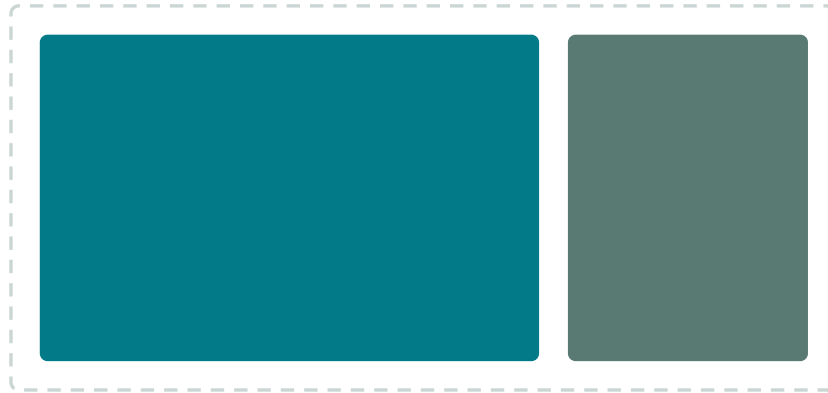
An escape hatch exists for the rare reverse case — max-md: applies below 768 — but mobile-first should be your default.

Stacked on a phone, side by side on a laptop



375 px

stacked



1024 px

two thirds / one third

Either answer earns full marks

With grid

```
<section class="grid md:grid-cols-3">  
  <article class="md:col-span-2">  
  <aside>
```

With flex

```
<section class="flex flex-col  
              md:flex-row">  
  <article class="md:w-2/3">  
  <aside class="md:w-1/3">
```

One class makes the switch happen. Task 3 asks you to point at it — be able to say which one and why.

It generates only the classes you actually used

The old way

Ship every class Tailwind could possibly make — a multi-megabyte stylesheet — then run a separate step to strip out everything the page did not use.

Just-in-time

Scan the source for class names, generate exactly those, on demand, as you type. Default since v3; in v4 the whole engine is built this way.

Three consequences you will actually feel

1 The CSS that ships is tiny

Only the classes you wrote exist in the output. A page like your HW3 one comes out a few kilobytes, not the whole framework.

2 Tailwind Play feels instant

The same compiler runs in the browser as you type. That is why the preview updates with no build and no refresh.

3 It reads your source as plain text

A class name assembled at runtime — `text-${color}-500` — is never generated, because the compiler never sees that string. Write the full class name out.

It is also why `p-[13px]` works at all: the class is made on demand. HW3 still does not allow it — the scale is what you are here to learn.

@theme turns your design tokens into utilities

```
<style type="text/tailwindcss">
  @theme {
    --color-brand: oklch(0.55 0.15 200);
  }
</style>

<button class="bg-brand text-white">Sign up</button>
```

Two things happen at once

- A family of utilities now exists: bg-brand, text-brand, border-brand, and the rest — Tailwind builds them from the token.
- It is also a real CSS variable on :root, so plain CSS and a style="" attribute can read the same token. One source of truth for the colour.

The namespaces you can set

--color-*

--font-*

--text-*

--spacing-*

--radius-*

--shadow-*

--breakpoint-*

In v4 the configuration lives in CSS. If a tutorial has you editing tailwind.config.js, you are reading about v3.

Custom colours work in Tailwind Play and with the CDN — try one. Custom breakpoints need a real build step.

Components made out of these same classes

Built on Tailwind

Every component is styled with utility classes — the same ones from today, no stylesheet of its own. Open a button and the class attribute is flexbox, spacing and breakpoints, exactly what you have been writing.

A component view

ui.shadcn.com/docs/components

Every component rendered live on its own page, in all its variants, with a Code tab showing the exact classes that produced it. Read it as a catalogue of how these properties get used in practice.

A real class string, from their button:

```
inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium
```

You copy the code into your project rather than install it, so those classes are yours to edit. That is the whole pitch: "the foundation for your design system," not a dependency you have to fight.

They are React components, which is where this course goes in Homework 4 and 5 — and what two of Tuesday's team presentations were about.

VS Code, two extensions, two drill files

The drills

`drills-flex` · `drills-grid`

Two files, sixteen containers with an empty class attribute and a goal. They grade themselves in the browser: fill in classes, save, watch the counter climb. Flexbox first, then grid. Open them in VS Code — they are whole documents with a script, so Tailwind Play cannot run them.

Tailwind Play

`play.tailwindcss.com`

Optional scratchpad for trying three versions of a card fast. It takes markup, not a whole file — so use it alongside VS Code, not instead of it, and remember the graded thing is the file in your repository.

VS Code extensions

`IntelliSense` + `Live Server`

IntelliSense autocompletes class names and shows the CSS a class compiles to when you hover it. Live Server reloads the browser on every save. Editor on one half of the screen, browser on the other: that is the loop.

Due Friday 9/25

1 Today: Do the drills

In VS Code: drills-flex.html until 9 of 9, then drills-grid.html until 7 of 7.

2 Task 1 — the header

flex, justify-between, items-center. A nested flex for the links.

3 Task 2 — the cards

grid, then grid-cols-1 md:grid-cols-2 lg:grid-cols-3.

4 Task 3 — the sidebar

Stacked below md, two thirds and one third above it. Grid or flex.

5 Screenshot and write up

phone.png at 375, desktop.png at 1280, five short answers, AI statement.